

✓ Actividad Operaciones de Vecindad, convoluciones y generar ruido en imágenes

Objetivos: Que el alumno aprenda a crear ruido en imagenes y realizar operaciones de vecindad.

✓ Ejercicio 1: Importar bibliotecas necesarias

Nombre del Alumno: Manuel Ramírez Galván

Numero de actividad: 2

Fecha: 21/02/2025

```
from google.colab import drive
drive.mount('/content/drive')
```

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

🔗 Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

✓ Ejercicio 2: Generación de Ruido Gaussiano

Objetivo: Aplicar ruido gaussiano a una imagen para simular perturbaciones típicas en sensores de cámaras digitales. **Pasos:**

1. Cargar la imagen en escala de grises usando `cv2.imread()`.
2. Generar ruido gaussiano, que se distribuye según una media y una desviación estándar.
3. Sumar el ruido a la imagen y asegurarse de que los valores estén en el rango `[0, 255]`.
4. Visualizar la imagen original y la imagen con ruido usando `plt.subplot()`.
5. Guarda la imagen con ruido usando `cv2.imwrite()`

```
image = "/content/drive/MyDrive/Colab Notebooks/2_1_Operaciones_de_vecindad_y_generacion_de_ruido_en_imágenes/data/Foto5.jpg"
image = cv2.imread(image, cv2.IMREAD_GRAYSCALE)
```

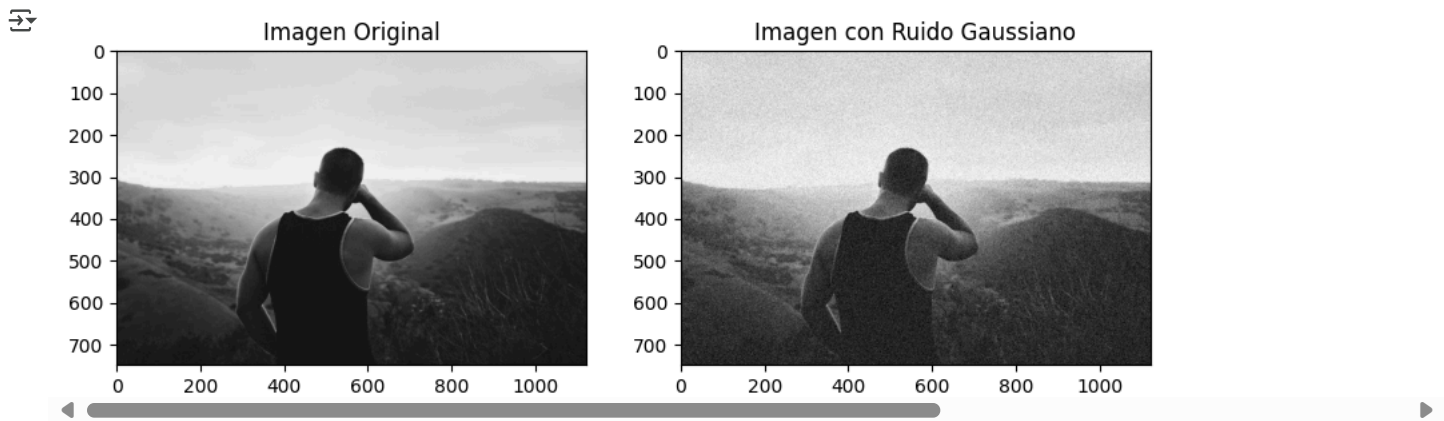
```
def add_gaussian_noise(image, mean=0, sigma=25):
    noise = np.random.normal(mean, sigma, image.shape).astype(np.float32)
    noisy_image = image.astype(np.float32) + noise
    return np.clip(noisy_image, 0, 255).astype(np.uint8)
```

```
image_gaussian = add_gaussian_noise(image)
```

```
plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.imshow(image, cmap='gray')
plt.title('Imagen Original')
```

```
plt.subplot(1, 2, 2)
noisy_image = add_gaussian_noise(image)
plt.imshow(noisy_image, cmap='gray')
plt.title('Imagen con Ruido Gaussiano')
```

```
plt.show()
```



✓ Ejercicio 3: Generación de Ruido Uniforme

Objetivo: Aplicar ruido uniforme, donde los valores aleatorios tienen la misma probabilidad dentro de un rango determinado.

Pasos:

1. Generar ruido uniforme, donde los valores se toman de una distribución uniforme en el rango $[-I, +I]$
2. Sumar el ruido a la imagen y asegurarse de que los valores estén en el rango $[0, 255]$.
3. Visualizar la imagen original y la imagen con ruido usando `plt.subplot()`.
4. Guarda la imagen con ruido usando `cv2.imwrite()`

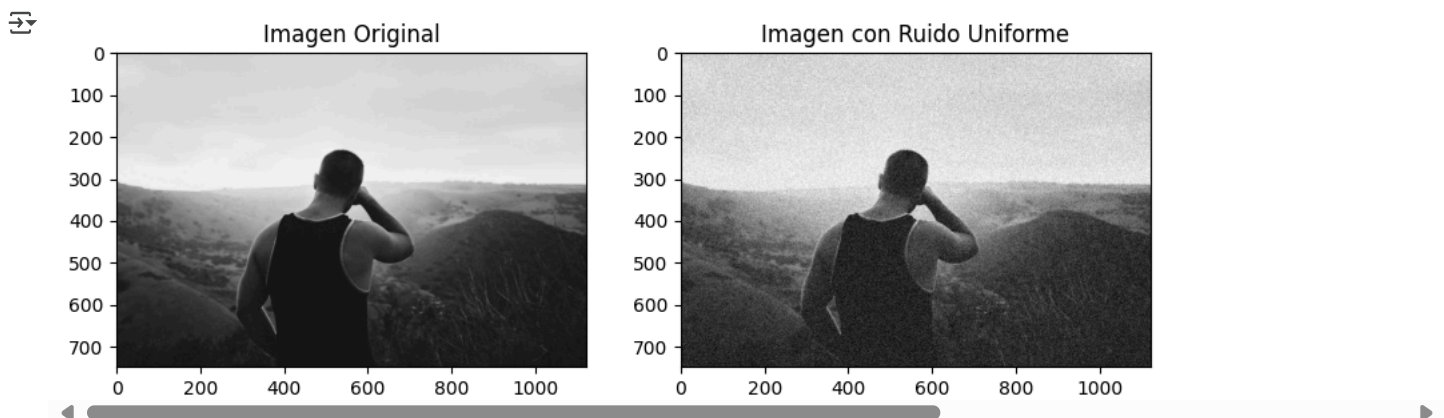
```
def add_uniform_noise(image, intensity = 50):
    noise = np.random.uniform(-intensity, intensity, image.shape).astype(np.float32)
    noisy_image = image.astype(np.float32) + noise
    return np.clip(noisy_image, 0, 255).astype(np.uint8)
```

```
image_uniform = add_uniform_noise(image)
```

```
plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.imshow(image, cmap='gray')
plt.title('Imagen Original')
```

```
plt.subplot(1, 2, 2)
plt.imshow(image_uniform, cmap='gray')
plt.title('Imagen con Ruido Uniforme')
```

```
plt.show()
```



✓ Ejercicio 4: Generación de Ruido Sal y Pimienta

Objetivo: Simular ruido de impulsos (sal y pimienta), que representa fallos en sensores o transmisión de datos.

Pasos:

1. Definir la probabilidad de ruido, indicando cuántos píxeles se modificarán.

2. Añadir píxeles blancos ("sal") y negros ("pimienta") aleatoriamente en la imagen.
3. Visualizar la imagen original y la imagen con ruido usando `plt.subplot()`.
4. Guarda la imagen con ruido usando `cv2.imwrite()`

```
def add_salt_and_pepper_noise(image, prob=0.02):
    noise_img = image.copy()
    num_pixels = int(prob * image.size)

    coords = [np.random.randint(0, i, num_pixels) for i in image.shape]
    noise_img[coords[0], coords[1]] = 255

    coords = [np.random.randint(0, i, num_pixels) for i in image.shape]
    noise_img[coords[0], coords[1]] = 0

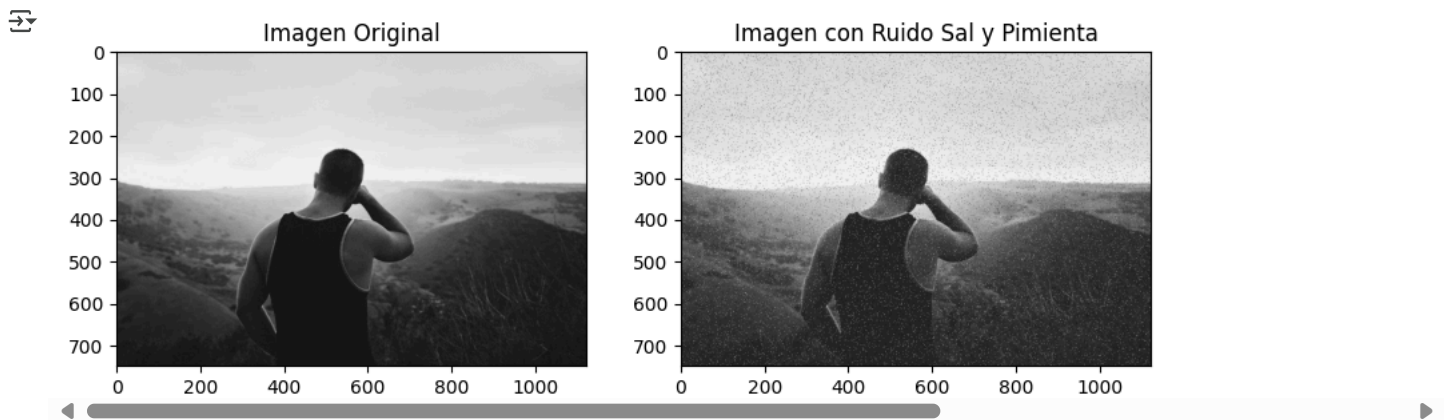
    return noise_img

image_salt_and_pepper = add_salt_and_pepper_noise(image)

plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.imshow(image, cmap='gray')
plt.title('Imagen Original')

plt.subplot(1, 2, 2)
plt.imshow(image_salt_and_pepper, cmap='gray')
plt.title('Imagen con Ruido Sal y Pimienta')

plt.show()
```



✓ Ejercicio 5: Relación de 4 Vecinos

Objetivo: Identificar los 4 vecinos de un píxel en una imagen.

Pasos:

1. Cargar la imagen en escala de grises `Tablero.png`.
2. Definir una función que devuelva los 4 vecinos de un píxel dado.
3. Ejecutar la función para un píxel específico y mostrar los resultados
4. Muestra la imagen con los vecinos marcados.

```
image2 = "/content/drive/MyDrive/Colab Notebooks/2_1_Operaciones_de_vecindad_y_generacion_de_ruido_en_imágenes/data/Tablero.png"
image2 = cv2.imread(image2, cv2.IMREAD_GRAYSCALE)
```

```
def get_4_neighbors(image, x, y):
    neighbors = [(x-1, y), (x+1, y), (x, y-1), (x, y+1)]
    return [(i, j) for i, j in neighbors if 0 <= i < image.shape[0] and 0 <= j < image.shape[1]]

x, y = 11, 6
neighbors = get_4_neighbors(image2, x, y)
print("4 vecinos:", neighbors)
```

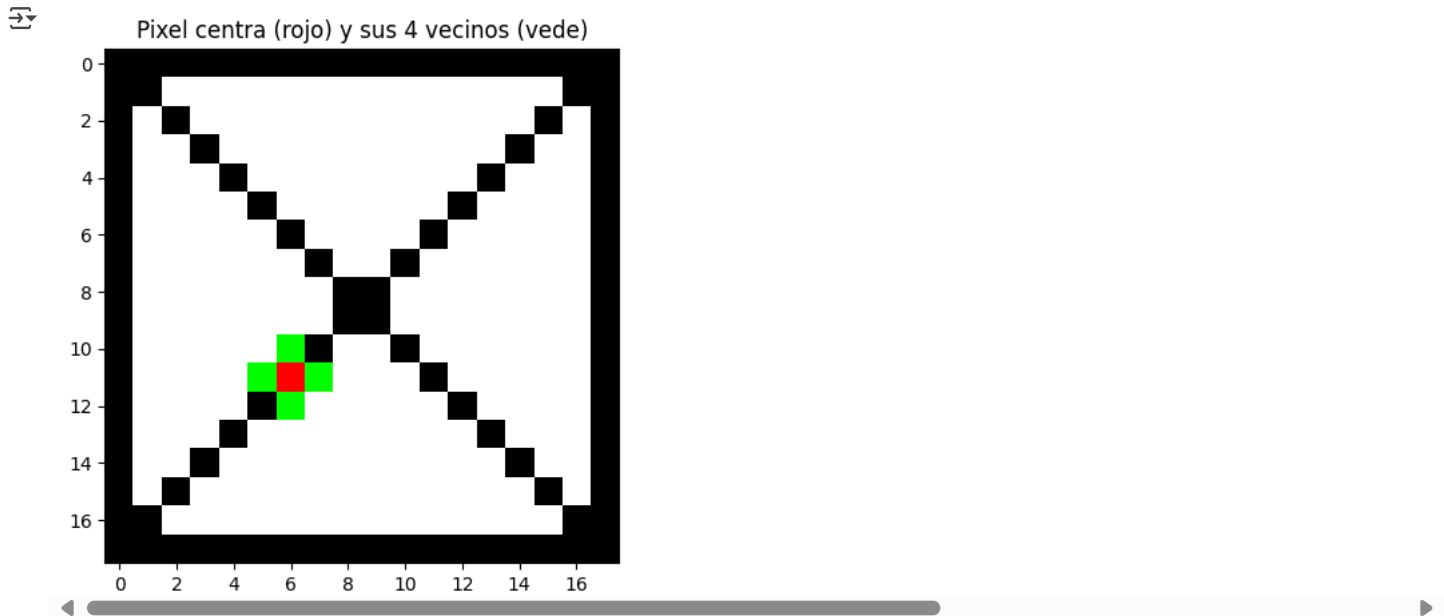
```
4 vecinos: [(10, 6), (12, 6), (11, 5), (11, 7)]
```

```
image_4 = cv2.cvtColor(image2, cv2.COLOR_GRAY2BGR)
```

```
image_4[x, y] = (0, 0, 255)

for i, j in neighbors:
    image_4[i, j] = (0, 255, 0)

plt.figure(figsize=(10, 5))
plt.imshow(cv2.cvtColor(image_4, cv2.COLOR_BGR2RGB))
plt.title("Pixel centra (rojo) y sus 4 vecinos (vede)")
plt.show()
```



✓ Ejercicio 6: Relación de 8 Vecinos

Objetivo: Identificar los 8 vecinos de un píxel en una imagen.

Pasos:

1. Definir una función que devuelva los 8 vecinos de un píxel.
2. Ejecutar la función para un píxel específico y mostrar los resultados.
3. Muestra la imagen con los vecinos marcados.

```
def get_8_neighbors(image, x, y):
    neighbors = [(x-1, y), (x+1, y), (x, y-1), (x, y+1), (x-1, y-1), (x-1, y+1), (x+1, y-1), (x+1, y+1)]
    return [(i, j) for i, j in neighbors if 0 <= i < image.shape[0] and 0 <= j < image.shape[1]]

x, y = 11, 6
neighbors = get_8_neighbors(image2, x, y)
print("8 vecinos:", neighbors)
```

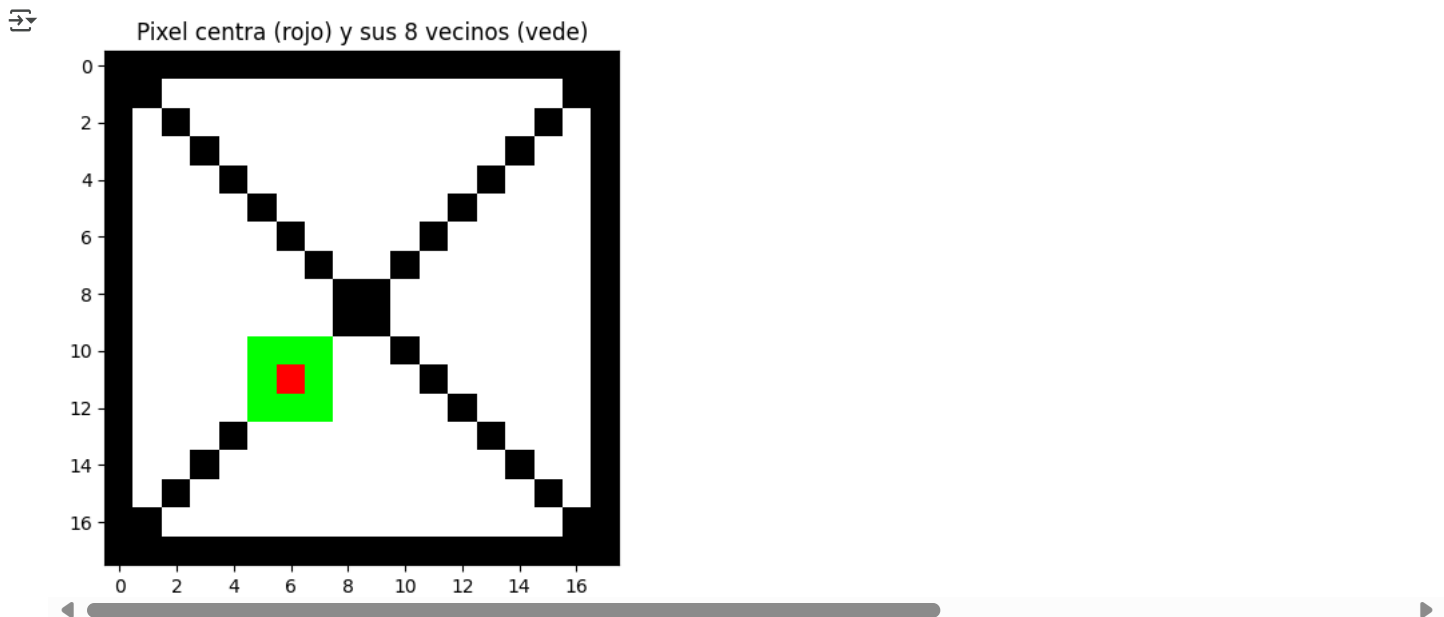
8 vecinos: [(10, 6), (12, 6), (11, 5), (11, 7), (10, 5), (10, 7), (12, 5), (12, 7)]

```
image_8 = cv2.cvtColor(image2, cv2.COLOR_GRAY2BGR)

image_8[x, y] = (0, 0, 255)

for i, j in neighbors:
    image_8[i, j] = (0, 255, 0)

plt.figure(figsize=(10, 5))
plt.imshow(cv2.cvtColor(image_8, cv2.COLOR_BGR2RGB))
plt.title("Pixel centra (rojo) y sus 8 vecinos (vede)")
plt.show()
```



✓ Ejercicio 7: Relación de 4 Vecinos en Diagonal

Objetivo: Identificar los 4 vecinos en diagonal de un píxel.

Pasos:

1. Definir una función que devuelva los 4 vecinos en diagonal de un píxel.
2. Ejecutar la función para un píxel específico y mostrar los resultados.
3. Muestra la imagen con los vecinos marcados.

Comienza a programar o [generar](#) con IA.

Comienza a programar o [generar](#) con IA.

Comienza a programar o [generar](#) con IA.

Comienza a programar o [generar](#) con IA.

Comienza a programar o [generar](#) con IA.

Comienza a programar o [generar](#) con IA.

Comienza a programar o [generar](#) con IA.

```
def get_diagonal_neighbors(image, x, y):
    neighbors = [(x-1, y-1), (x-1, y+1), (x+1, y-1), (x+1, y+1)]
    return [(i, j) for i, j in neighbors if 0 <= i < image.shape[0] and 0 <= j < image.shape[1]]
x, y = 11, 6
neighbors = get_diagonal_neighbors(image2, x, y)
print("vecinos diagonal:", neighbors)
```

vecinos diagonal: [(10, 5), (10, 7), (12, 5), (12, 7)]

```
image_4d = cv2.cvtColor(image2, cv2.COLOR_GRAY2BGR)
```

```
image_4d[x, y] = (0, 0, 255)
```

```
for i, j in neighbors:
    image_4d[i, j] = (0, 255, 0)
```

```
plt.figure(figsize=(10, 5))
plt.imshow(cv2.cvtColor(image_4d, cv2.COLOR_BGR2RGB))
plt.title("Pixel centra (rojo) y sus 4 vecinos en diagonal (vede)")
plt.show()
```

```
plt.show()
```



Pixel centra (rojo) y sus 4 vecinos en diagonal (vede)

